

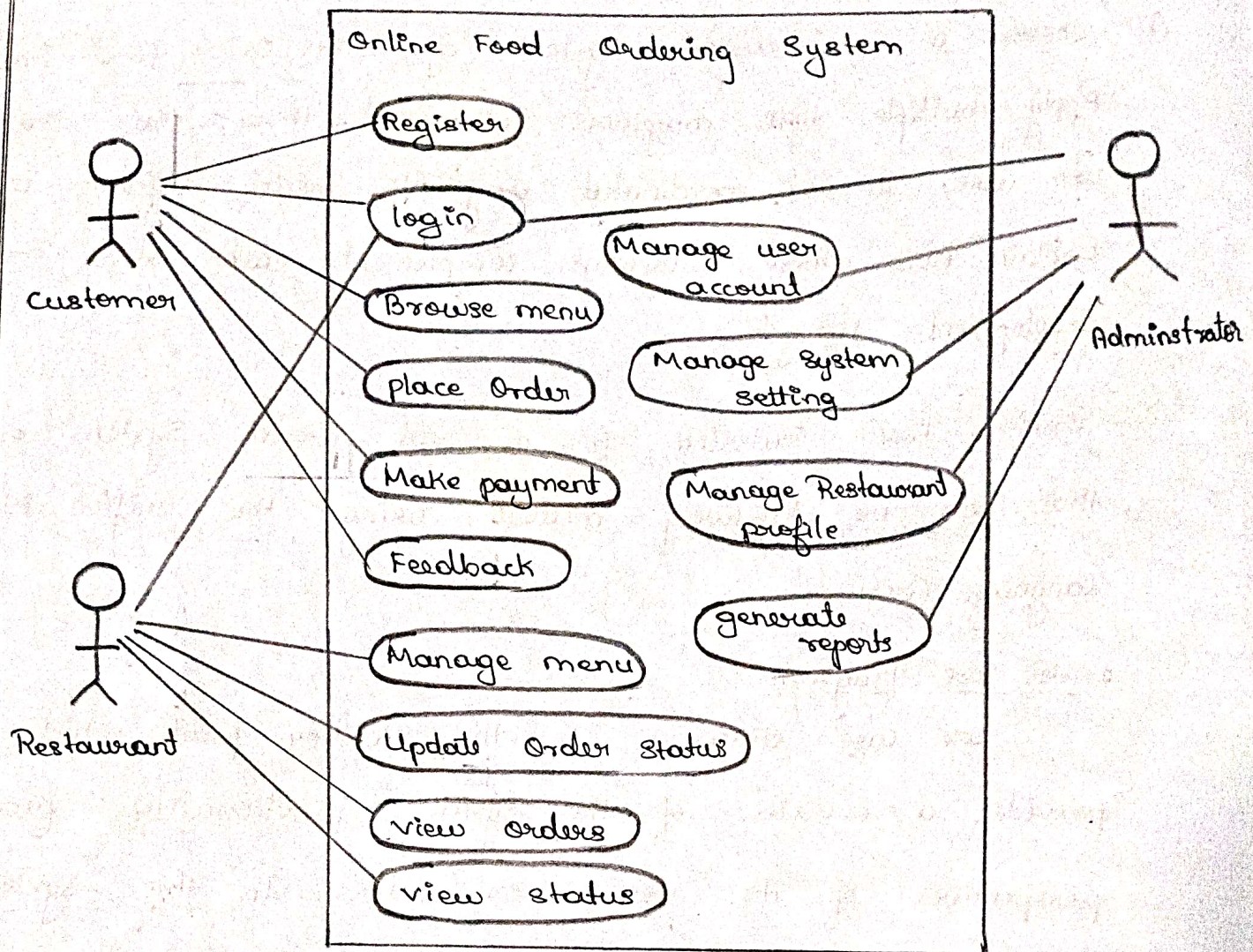
- ① Choose a real-world System (eg., ATM, Online Food Ordering). Apply multiple UML diagrams (at least three: class, sequence, use case) in a coordinated way to model different aspects. Explain how these diagrams complement each other in the development Lifecycle.

⇒ Online Food Ordering is a real world System example that can be effectively modeled using the unified Modeling language (UML).

1. Use Case Diagram

Use Case diagram is the starting point, which will provide a overview of the system's functionality from the perspectives of its users, or actors. In this system, the main actors are the customer, Restaurant and Administrator.

- * Customer : Interacts with the System to Browse Menu, Select Food Items, Place order and make payment.
- * Restaurant : Manages the menu and receives orders.
- * Administrator : Handles back end - management task like Manage users and generate reports.



2]. class Diagram :

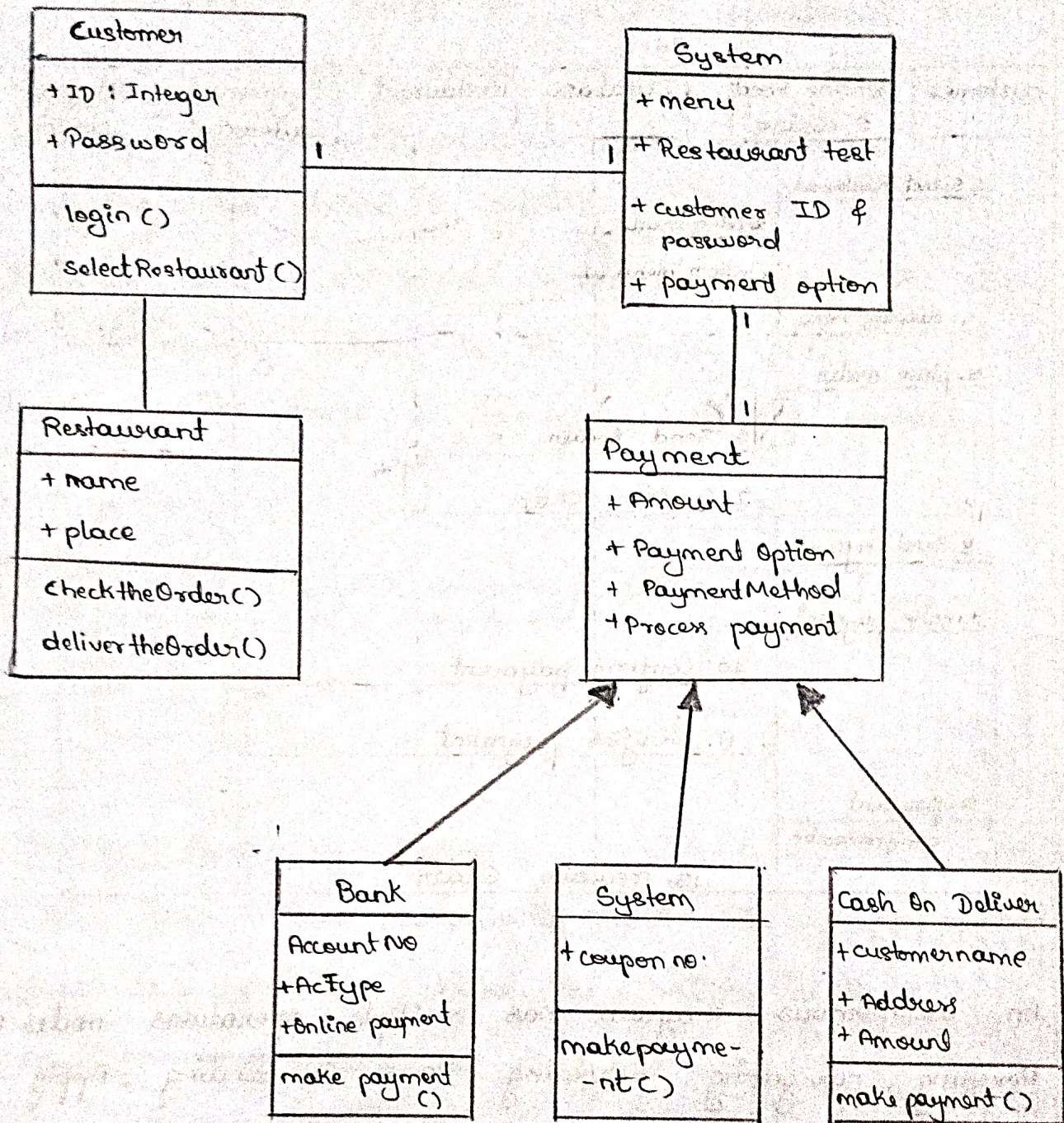
It will provide structural view of the system. In the class diagram, they consists of attributes, methods and relationships between them.

* Customer : Attributes like name, address, Contact. Methods like placeOrder().

* System : Attributes are menu Restaurant.

* Restaurant : name and place of Restaurant.

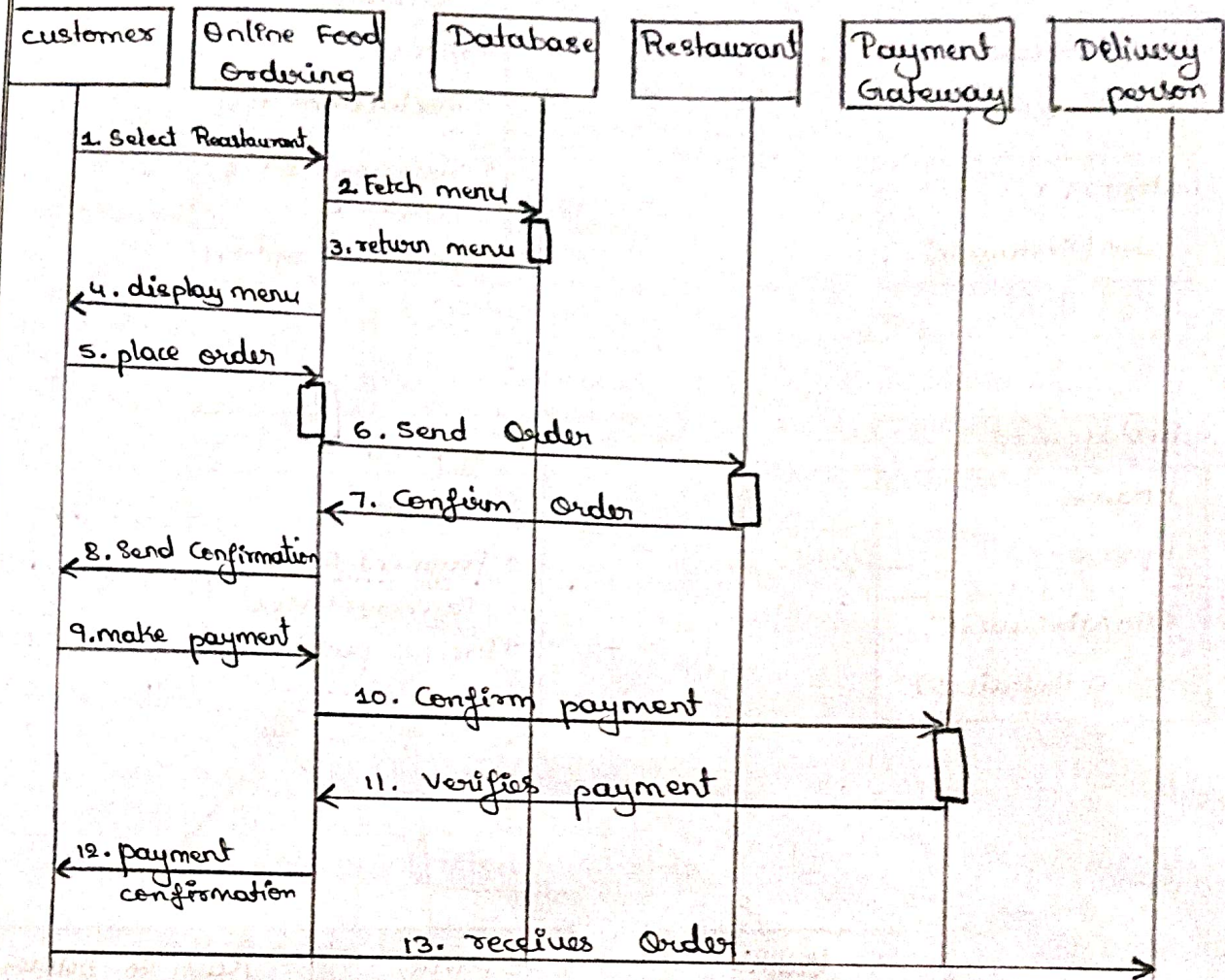
In this class diagram, they may have different classes and each class consists different attributes and methods.



3] Sequence diagram

It outlines the process flow of an online ordering system. It begins with the customer selecting a restaurant, triggering the Online Food Ordering System to fetch the menu from the Restaurant data base once retrieved the System displays the menu to the customer. The customer then places an order which is sent to the restaurant. After confirmation customer makes payment.

After Confirms payment , they receives Confirmation . PAGE 4
 Finally , the order receives through delivery person .



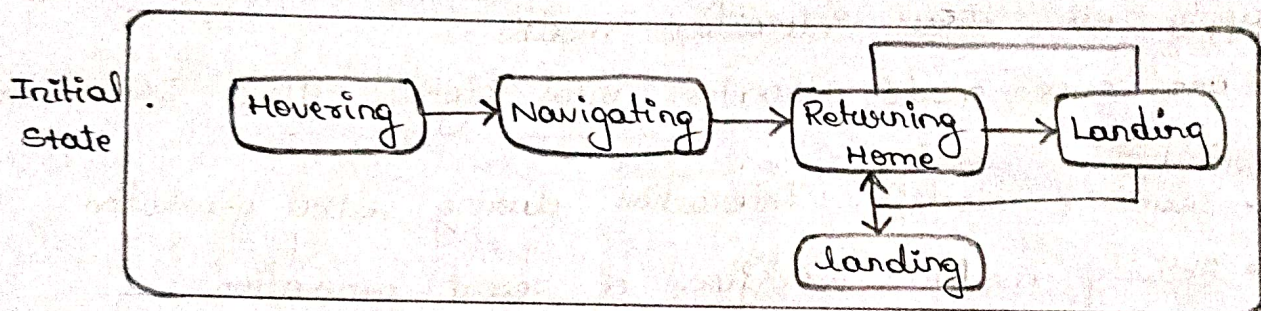
2] An autonomous diagram has multiple operations modes :
 Hovering , navigating , Returning Home & landing . Apply nested
 & concurrent states to model flight control & battery monitoring
 as parallel process , Show that how a low battery signal
 trigger a transitions from navigating to Returning Home .

⇒ UML State Machine diagram models the autonomous
 drone's behavior using concurrent states to handle 2
 Independent processes : Flight Control and Battery monitoring .

1) Flight Control Region

This region contains the primary operational modes of the drone, which are modeled as

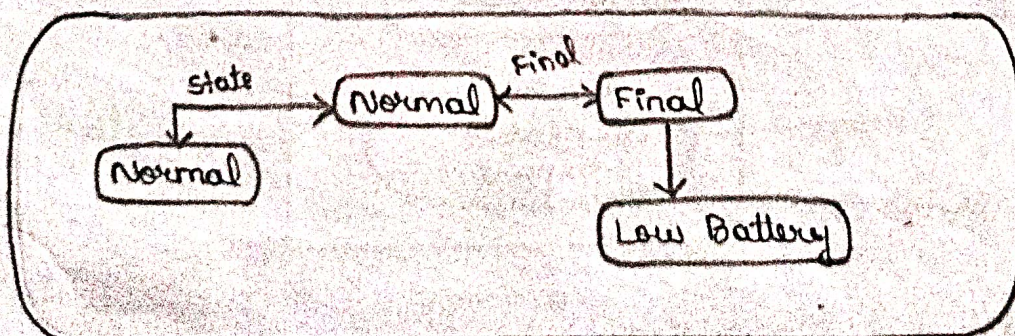
- **Hovering** : The drone is airborne but stationary.
- **Navigating** : The drone is activity moving.
- **Returning Home** : A special state entered when the drone needs to autonomously return to its starting point.
- **Landing** : The drone is descending to the ground.



2. Battery Monitoring Region

This Region runs concurrently with the flight control region. It constantly checks the battery level.

- **Normal** : The battery has sufficient charge.
- **Low Battery** : The battery level has dropped between a critical threshold.



Low Battery Signal Triggers a Transition

PAGE 6

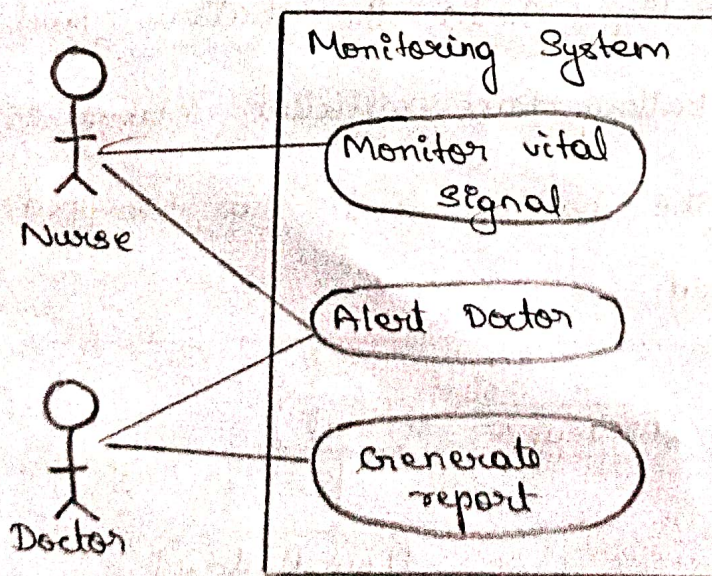
A transition from navigating is Returning Home is triggered by a guard condition originating from the concurrent Battery monitoring region. When the Battery monitoring region transitions to the Low Battery state, it sends a signal. This signal is received by the flight control region and evaluated as a trigger.

3] Use a hospital patient monitoring system as an example, apply all three interaction models:

- use case model: Monitor vital signals. Alert Doctor, Generate report.
- sequence model: Interaction during alert generation.
- Activity model: workflow of report generation.

Explain how these models collectively capture system behaviour and support team collaboration during development.

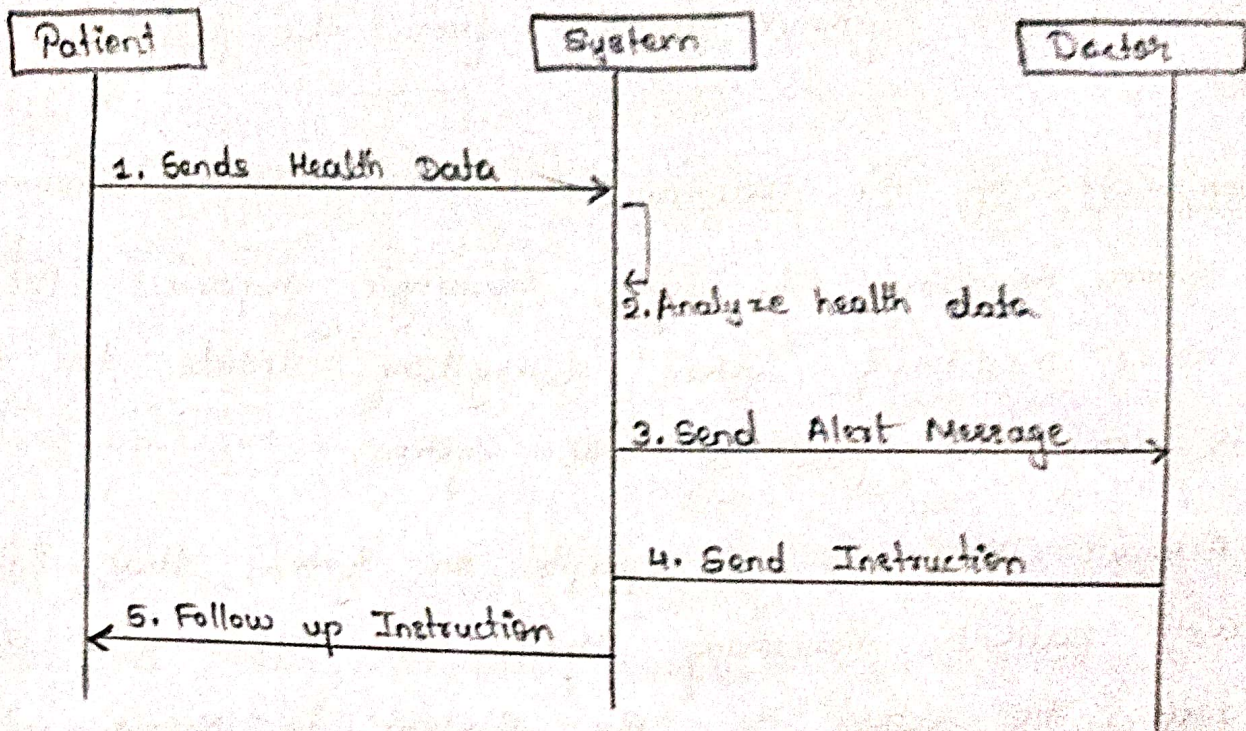
Use case Model: Hospital Patient



UML use case model, which provides a high level PAGE 7
view of the system's function from the perspective of the user.

Component of the Diagram:

- System Boundary: The large rectangle represents the "Hospital Patient Monitoring System". Everything inside the boundary is considered part of the system.
- Actors: These are the people or system that interact with patient monitoring system to achieve a specific goal. The actors in the system is Nurse - who actively monitor the patient's condition and doctor - who interacts with the system to receive alerts about critical conditions and to receive patient data.
- use cases: The ovals inside the system boundary represent the major functions or goals of the system.
 - Monitor vital signals - It involves the system collecting and displaying data from a patient.
 - Alert doctor - This is a function that is triggered when a patient's vital signs are outside of a safe range.
 - Generate report - This involves compiling the collected vital sign data over a period of time to create a summary report.

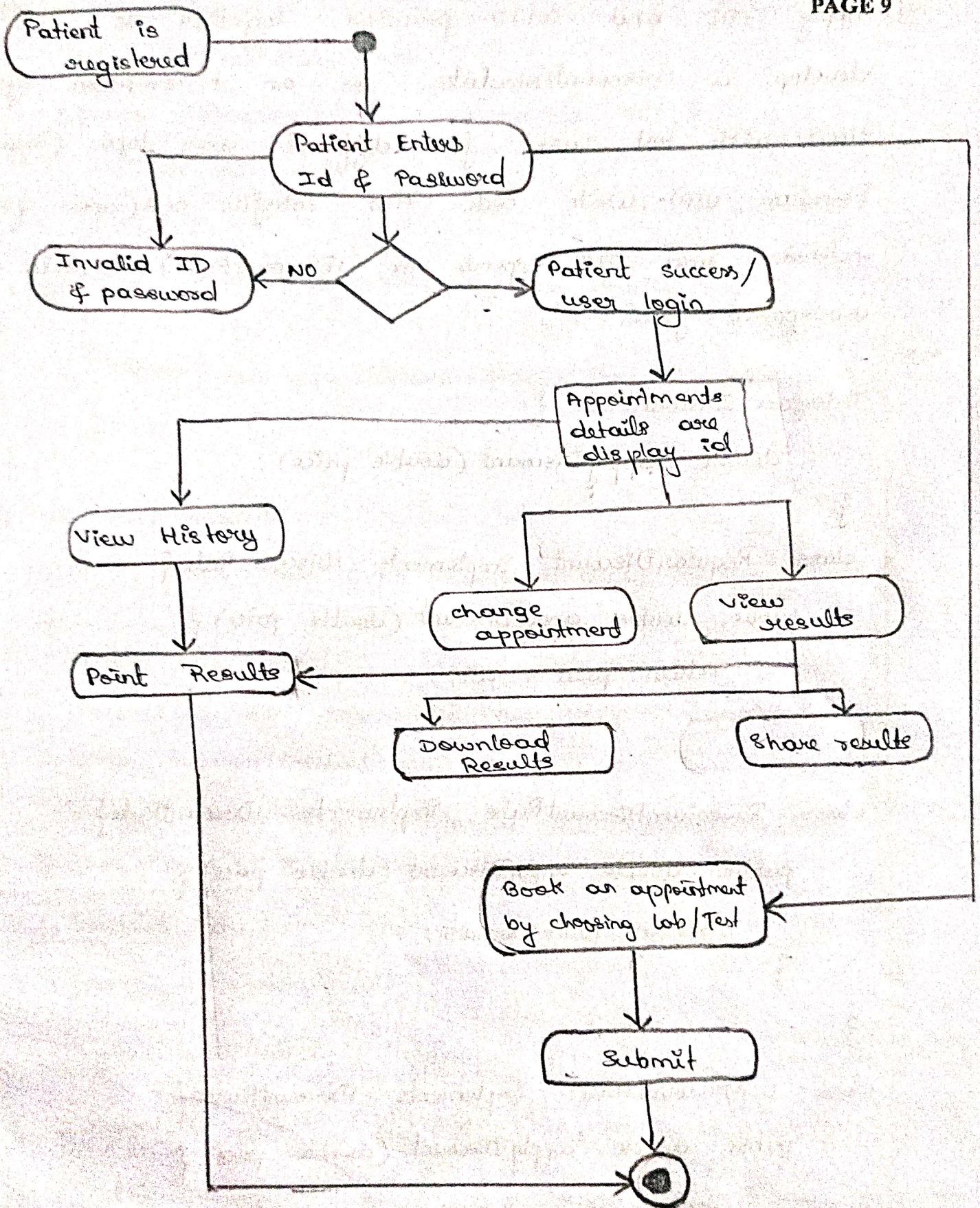


Sequence Model for a patient monitoring System illustrates the dynamic, time-ordered interaction between different objects to achieve a specific goal in this alert generation. The patient sends health data to the System. The System will analyze the health data if any variation occurs the System will send alert message to the doctor. The doctor will send the instruction based on certain variation, and the System will notify the patient to follow up instruction.

Activity Model :

Team collaboration Benefits : Business Analyst use the use case model to gather and confirm requirement.

- Developer rely on the sequence model to implement precise interaction logic



4] Apply TDD and SOLID principles together to PAGE 10
develop a DiscountCalculator for an e-commerce system.
Start with test cases for different user types (Regular,
Premium, VIP). Write code that satisfies OCP (open for
extension) and DIP (Depends on IDiscountRule). Explain the
development flow.

⇒

```
Interface IDiscountRule {
```

```
    double applyDiscount (double price);
```

```
}
```

```
class RegularDiscount implements IDiscountRule {
```

```
    public double applyDiscount (double price) {
```

```
        return price * 0.90;
```

```
    }
```

```
}
```

```
class PremiumDiscountRule implements IDiscountRule {
```

```
    public double applyDiscount (double price) {
```

```
        return price * 0.85;
```

```
    }
```

```
}
```

```
class VIPDiscountRule implements IDiscountRule {
```

```
    public double applyDiscount (double price) {
```

```
        return price * 0.75;
```

```
    }
```

```
}
```



```

class DiscountCalculator {
    private IDiscountRule rule;

    public DiscountCalculator (IDiscountRule rule) {
        this.rule = rule;
    }

    public double calculatePrice (double OriginalPrice)
    {
        return rule.applyDiscount (OriginalPrice);
    }
}

public class Main {
    public static void main (String[] args)
    {
        double price = 200.0;
        DiscountCalculator regular = new DiscountCalculator
        (new RegularDiscountRule());
        DiscountCalculator premium = new DiscountCalculator
        (new PremiumDiscountRule());
        DiscountCalculator vip = new DiscountCalculator
        (new VIPDiscountRule());
        System.out.println ("Regular : " + regular.calculatePrice (price));
        System.out.println ("Premium : " + premium.calculatePrice (price));
        System.out.println ("VIP : " + vip.calculatePrice (price));
    }
}

```


SOLID is Applied

S - Single Responsibility Principle

Each class has one job.

O - open / closed Principle

To add "StudentDiscountRule" just create a new class implementing IDiscountRule

Discount Calculator does not need to change.

L - Liskov Substitution Principle

Any IDiscountRule can replace another without breaking DiscountCalculator.

I - Interface Segregation Principle.

IDiscountRule is small focused Interface.

D - Dependency Inversion Principle

Discount Calculator depends on the IDiscountRule abstraction, not concrete discount classes.

5] A banking application needs to integrate a legacy Mainframe Account Validator, that uses a different data format. Apply the Adapter pattern to wrap it and make it compatible with the modern Account validation service. Then, explain how this supports open-closed principles by allowing new adapters without modifying client code.

Interface AccountValidationService {

PAGE 13

boolean isValid (Account account);

}

class Account {

String accountNumber;

String holderName;

public Account (String accountNumber, String holder
name) {

 this.account Number = accountNumber;

 this.holder Name = holderName;

}

}

class legacy Main {

public boolean check Account (String legacyData)

{

 return legacyData != null && legacy Data . starts
 with ("ACC");

}

}

class Mainframe implements AccountValidation Service {

private legacy MainframeAccount Validation legacyValidator;

public Mainframe (legacyMainframeAccount Validator legacyValidator)

{

 this.legacyValidator = legacyValidator;

}

@Override


```

public boolean isValid (Account account)
{
    String legacyData = account.getAccountNumber() + account
    holderName;
    return legacyValidator.checkAccount (legacyData);
}
}

public class Main {
    public static void main (String[] args) {
        Account account = new Account ("ACC12345", "Arya",
        AccountValidationService Validator = new
        Mainfram (new legacy MainframAccountValidator ());
        System.out.println ("is account valid?" + validator.isValid
        (account));
    }
}

```

Open for extension:

If then moreover we have another legacy system, we can create a new adapter that implements AccountValidationService without touching the client code.

closed for Modification:

The client always uses the AccountValidationService interface, so it never changes only new adapter classes are added,